
Section C

You are advised to spend no more than **65 minutes** on this section.

Enter your answers to **Section C** in your Electronic Answer Document. You **must save** this document at regular intervals.

These questions require you to load the **Skeleton Program** and to make programming changes to it.

1	1
---	---

This question refers to the subroutine `SendMorseCode`.

What you need to do:**Task 1**

Modify the subroutine `SendMorseCode` so that it checks each character of the input string entered by the user.

If a character is not an uppercase letter or a space then the program should use the subroutine `ReportError` to output a suitable error message.

The subroutine should then set `MorseCodeString` to the empty string and exit.

Task 2

Test that the changes you have made work by conducting the following test:

- run the program
- choose option `S`
- enter the string `Help`

Evidence that you need to provide

Include the following evidence in your Electronic Answer Document.

1	1	.	1
---	---	---	---

Your amended PROGRAM SOURCE CODE for the subroutine `SendMorseCode`.

[4 marks]

1	1	.	2
---	---	---	---

SCREEN CAPTURE(S) showing the requested test.

[1 mark]

Turn over for the next question

Turn over ►

1 2

This question will extend the functionality of the program. To send a Morse code message, the dots and dashes need to be converted into signals.

What you need to do:

Task 1

Create a new subroutine `SendSignals` that converts a Morse code message such as

-△.△.-△△△-..-

(TEA X) to its equivalent signal

===△△△=△△△=△===△△△△△△===△=△=△===△

by using the rules

- a dot is converted to an equals sign (=) followed by a space
- a dash is converted to three equals signs (===) followed by a space
- a space is converted to two spaces.
(This results in three spaces after the signals for a letter.)

Note that △ represents a space and should be displayed as a space on screen.

The subroutine is to output the complete transmission string to the screen.

Add the call `SendSignals(MorseCodeString)` as the last statement to be executed in the subroutine `SendMorseCode`.

Task 2

Test that the changes you have made work by conducting the following test:

- run the program
- choose option S
- enter the string MORSE X

Evidence that you need to provide

Include the following evidence in your Electronic Answer Document.

1 2

. 1 Your PROGRAM SOURCE CODE for the subroutine `SendSignals`.

[7 marks]

1 2

. 2 SCREEN CAPTURE(S) showing the Morse code message and the signals.

[1 mark]

1 3

This question will extend the program to output the letters of the alphabet and their corresponding Morse codes.

What you need to do:

Task 1

Create a new subroutine `OutputAlphabetWithCode`. This subroutine is to display each uppercase letter of the alphabet and its corresponding Morse code. The output is to be created using the data in `Letter` and `MorseCode`.

The format of the output should be as shown in **Figure 4**.

- Each uppercase letter is followed by a space and then its Morse code, grouped four to a line.
- Each Morse code should be padded with trailing spaces to a width of 6 characters.

Figure 4

A .-	B -...	C -.-.	D -..
E .	F ..-.	G --.	H
I ..	J .---	K -.-	L .-..
M --	N -.	O ---	P .--.
Q --.-	R .-.	S ...	T -
U ..-	V ...-	W .--	X -.-.
Y -.-.	Z --..		

Task 2

Amend the subroutines `DisplayMenu` and `SendReceiveMessages` so that the user can choose a new option A that will call `OutputAlphabetWithCode`.

Task 3

- run the program
- choose new option A

Evidence that you need to provide

Include the following evidence in your Electronic Answer Document.

1 3**1**

Your PROGRAM SOURCE CODE for the subroutine `OutputAlphabetWithCode`. Your amended PROGRAM SOURCE CODE for the subroutines `DisplayMenu` and `SendReceiveMessages`.

[6 marks]

1 3**2**

SCREEN CAPTURE(S) showing the main menu and the output from the test run.

[1 mark]

Turn over for the next question

Turn over ►

1	4
---	---

This question will further extend the functionality of the program.

Before sending a message in Morse code the message is to be encrypted using a Caesar cipher. To encrypt a character (letter or space) it is shifted a given number of places (the key) in the alphabet (space is taken as the first character, then A, B, C and so on to Z). If a character is required before space or after Z then wrap-round is to be implemented.

For example:

- if the key is 2, then SPACE is encrypted as B, A is encrypted as C, B is encrypted as D, ..., X is encrypted as Z, Y is encrypted as SPACE, and Z is encrypted as A.
- if the key is -1, then SPACE is encrypted as Z, A is encrypted as SPACE, B is encrypted as A, ..., and Z is encrypted as Y.

Three different keys are to be used depending on the position of the character in the message.

The first character in the message is to be encrypted using the first key, the second character is encrypted with the second key and the third character with the third key.

The fourth character is then encrypted using the first key again, the fifth character is encrypted with the second key and the sixth character with the third key and so on.

The three keys are repeatedly used like this until the end of the message.

What you need to do:

Task 1

At the start of the subroutine `SendReceiveMessages` add code to enter three values that **must** be integers (the keys).

Task 2

Amend the subroutine `SendMorseCode` to encrypt the message using the substitution method described above and the keys entered by the user.

Task 3

Test that your code works by conducting the following test:

- run the program
- for the first key enter 17
- for the second key enter 5
- for the third key enter -3
- enter S
- enter the string TEA X

Evidence that you need to provide

Include the following evidence in your Electronic Answer Document.

1	4
---	---

.

1

 Your amended PROGRAM SOURCE CODE for the subroutines `SendMorseCode` and `SendReceiveMessages`.

[9 marks]

1	4
---	---

.

2

 SCREEN CAPTURE(S) showing the keys entered, the plain text and the encrypted Morse code message.

[1 mark]

END OF QUESTIONS